

Script-Controlled ACL

Restrict Record Access Based on Field Value

Project Documentation

1. Introduction

- **Project Title:** Script-Controlled ACL – Restrict Record Access Based on Field Value
- **Team Members:**
 - Arvindeshkiran Senthilkumar – Team Lead (ACL-Create)
 - Madhan S – Member (Project Outcome)
 - Pandian M – Member (ACL-Write, ACL-Delete)
 - Dharmesh R – Member (Users Creation, Roles Creation)
 - Jayabalakumaran R – Member (Creation of Table, ACL Configuration)

2. Project Overview

- **Purpose:** To control who can access a record based on the value stored in one of its fields. Instead of relying only on roles, script-controlled Access Control Lists (ACLs) evaluate the field value at run time, so a user can read, create, edit or delete a record only when the field value meets the defined condition. This protects sensitive records from unauthorized users.
- **Features:**
 - Creation of users and roles to represent different levels of access.
 - A custom table with the field on which record access is controlled.
 - Script-controlled ACLs for the Read, Create, Write and Delete operations.
 - Records whose field value does not satisfy the script condition are hidden or blocked for the user.
 - Verification of every ACL by impersonating users with different roles.

3. Architecture

This project is built on the ServiceNow platform, so the MERN layers are mapped to the platform components below.

- **Frontend:** The ServiceNow user interface (list and form views, Application Navigator). Users see or do not see records, and can or cannot create, edit or delete them, depending on the ACL result.
- **Backend:** Server-side ACL scripts written in JavaScript. Each script uses the current record (current) and the logged-in user (gs) to evaluate the field value and sets the answer variable to true (allow) or false (deny).
- **Database:** The custom table and its restricting field, together with the User (sys_user), Role (sys_user_role) and ACL (sys_security_acl) tables. ACL scripts read the field value from the record at the time of each request.

4. Setup Instructions

- **Prerequisites:**
 - A ServiceNow instance (for example, a Personal Developer Instance) with administrator access.
 - The security_admin role, obtained by elevating privileges, to create and edit ACLs.
 - A modern web browser (Chrome, Edge or Firefox).
- **Installation:**
 1. Log in to the instance as an administrator.
 2. Create the users and roles (Milestone 1) and assign the roles to the users.
 3. Create the custom table and the field used to restrict access (Milestone 2).
 4. Elevate privileges to security_admin.
 5. Create the script-controlled ACLs for Read, Create, Write and Delete on the table (Milestones 3 to 6).
 6. Test each ACL by impersonating the users (Milestone 7).

5. Folder Structure

ServiceNow stores configuration as records instead of project folders. The components are organized as follows.

- **Client:** Table list and form views, the Application Navigator module for the table, and the User and Role lists used to manage access.
- **Server:** ACL records under System Security > Access Control (ACL), one for each operation, each containing its ACL script.

6. Running the Application

- Open the instance URL and log in with the required user.
 - **Frontend:** Open the table list from the Application Navigator to view, create, edit or delete records.
 - **Backend:** ACL scripts run automatically on the server every time a record is accessed; no separate server needs to be started.
- Use Impersonate User to switch between users and check the behaviour of each ACL.

7. API Documentation

The application does not add new endpoints. The ACLs protect the standard table operations, including the REST Table API (for example, GET /api/now/table/<table_name>), which enforces the same ACLs as the user interface. The ACLs created are summarized below.

Milestone	Operation	Script condition	Result when false
3	Read	Allow only when the record field value satisfies the condition	Record is not visible
4	Create	Allow only when the field value on the new record satisfies the condition	Create is blocked

Milestone	Operation	Script condition	Result when false
5	Write	Allow only when the field value allows the update	Edit is blocked
6	Delete	Allow only when the field value allows deletion	Delete is blocked

Example ACL script:

```
(function () {
    // Allow access only when the field value meets the condition
    if (current.<field_name> == "<allowed_value>" ||
    gs.hasRole("<role_name>"))
        return true;
    return false;
})();
```

8. Authentication

- Users authenticate with their ServiceNow credentials and are tracked by a session; REST calls use the same user identity.
- Authorization is handled by roles and ACLs. A user must have the required role and pass the ACL script before the operation is allowed.
- For each operation, ServiceNow evaluates the ACLs on the table. If no ACL grants access, the request is denied.
- ACLs are created and edited only after elevating to the security_admin role.

9. User Interface

- Users and Roles lists showing the created users and their assigned roles.
- Table list and form view showing the field used to restrict access.
- ACL form with the script for the Read, Create, Write and Delete operations.
- Record list as seen by different users, showing the restricted records hidden.

[Insert screenshots of the above screens here]

10. Testing

Testing is done manually by impersonating each user and repeating every operation against records with different field values. The Debug Security Rules option can be used to see which ACL allowed or denied an action.

Operation	User	Field value	Expected result
Read	User with access	Meets the condition	Record visible
Read	User without access	Does not meet the condition	Record hidden
Create	User with access	Meets the condition	Record created
Write	User without access	Does not meet the condition	Update blocked

Operation	User	Field value	Expected result
Delete	User without access	Does not meet the condition	Delete blocked

11. Known Issues

- An error in an ACL script causes access to be denied, so scripts must be tested carefully.
- Users with the admin role may bypass an ACL unless the Admin overrides option is turned off.
- Only table-level ACLs are covered; field-level restrictions are not part of this project.
- Complex scripts can slow down list views on tables with many records.

12. Future Enhancements

- Add field-level ACLs to hide or protect individual fields.
- Move the condition logic into a Script Include so it can be reused across ACLs.
- Store the allowed field values in a system property so they can be changed without editing scripts.
- Add automated tests using the Automated Test Framework.
- Log denied access attempts for auditing.